



Smart Mall ●

Smart . Secure . Stylish

FINAL YEAR PROJECT

Documentation & Implementation

PROJECT REPORT

Submitted by:

[Student Full Name]

[Student ID / Registration Number]

Department of Computer Science

[University Name]

May 2026

DECLARATION

I, [Insert Your Full Name], hereby declare that this project report titled "Smart Mall: A Full-Stack E-Commerce Platform with Mobile Application for the Ethiopian Market" is my original work. I confirm that:

This work has not been submitted previously for any degree or qualification at any other institution.

All sources of information have been properly acknowledged and referenced.

The code, design, and implementation presented in this report are my own work unless otherwise credited.

Where the work of others has been used, it has been duly attributed through citation and references.

I understand that plagiarism is a serious academic offense and that the university reserves the right to take appropriate action if any part of this work is found to be plagiarized.

Name: [Insert Your Full Name]

Student ID: [Insert Student ID]

Date: [Insert Date]

Signature: _____

This declaration is made in accordance with the university's academic integrity policy.

APPROVAL PAGE

This project report entitled "Smart Mall: A Full-Stack E-Commerce Platform with Mobile Application for the Ethiopian Market" has been read and approved as meeting the requirements for the award of the degree of Bachelor of Science in Computer Science.

Supervisor: [Insert Supervisor Name]

Signature: _____

Date: [Insert Date]

Internal Examiner: [Insert Internal Examiner Name]

Signature: _____

Date: [Insert Date]

External Examiner: [Insert External Examiner Name]

Signature: _____

Date: [Insert Date]

Department Head: [Insert Department Head Name]

Signature: _____

Date: [Insert Date]

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my supervisor, [Insert Supervisor Name], for their invaluable guidance, constructive feedback, and continuous support throughout the development of this project. Their expertise and encouragement have been instrumental in shaping this work.

I am also grateful to the Department of Computer Science at [Insert University Name] for providing the necessary resources and academic environment that made this project possible. The knowledge and skills acquired through the coursework provided a solid foundation for undertaking this project.

Special thanks go to my family and friends for their unwavering support, patience, and understanding during the many hours spent researching, coding, and writing this report. Their encouragement kept me motivated throughout the challenges encountered during development.

Finally, I would like to acknowledge the developers of the open-source technologies used in this project, including PHP, MySQL, Apache, Capacitor, and the many libraries and tools that made this work feasible. The open-source community's contributions have been invaluable.

Name: [Insert Your Full Name]

Date: [Insert Date]

Place: [Insert Location]

ABSTRACT

E-commerce has transformed global retail, yet the Ethiopian market remains underserved by locally adapted online shopping platforms. Most existing solutions lack support for the Ethiopian Birr (ETB), fail to integrate with local payment gateways, and do not provide dedicated mobile applications for the growing number of smartphone users in the country. This project presents Smart Mall, a full-stack e-commerce platform designed specifically for the Ethiopian market, addressing these gaps through a comprehensive technical solution.

Smart Mall is built using PHP 8.0+ and MySQL (via PDO) for the backend, with HTML5, CSS3, and vanilla JavaScript for the frontend. The platform features a complete product catalog with category filtering, a server-side shopping cart with stock validation, a secure checkout system supporting three payment methods (Chapa Pay, Bank Transfer, and Cash on Delivery), and a comprehensive admin panel for store management. A key innovation is the multi-currency engine that displays prices in both USD and ETB using live exchange rates fetched from the ExchangeRate-API, with file-based caching for optimal performance.

The system includes a companion mobile application built with Capacitor v8, wrapping the web platform into a native Android app using Android Studio. The mobile app features immersive full-screen browsing, hardware back-button navigation, and deep-linking support for Chapa payment callbacks. Security is implemented through CSRF token verification on all POST operations, PDO prepared statements to prevent SQL injection, bcrypt password hashing, and htmlspecialchars output escaping for XSS prevention.

The platform was developed locally using XAMPP and deployed on FreeProHost shared hosting for production. Testing covered functional testing of all user flows, payment verification testing in Chapa sandbox mode, mobile app testing on Android emulator and physical devices, and security testing including CSRF bypass attempts, SQL injection probes, and XSS attack vectors.

Keywords: E-Commerce, PHP, MySQL, Chapa Payment Gateway, Capacitor, Android, Multi-Currency, Ethiopia, Full-Stack, Mobile Application, PWA

TABLE OF CONTENTS

(Right-click here in Word > Update Field to auto-generate the Table of Contents)

LIST OF FIGURES

Figure: System Architecture Diagram

Figure: Use Case Diagram

Figure: Data Flow Diagrams (Level 0 and 1)

Figure: Entity-Relationship Diagram

Figure: Navigation Flow Diagram

Figure: Homepage - Product Grid

Figure: Product Detail Page

Figure: Shopping Cart

Figure: Checkout Page

Figure: Order Confirmation

Figure: User Registration

Figure: User Login

Figure: Admin Dashboard

Figure: Admin Add Product Form

Figure: Admin Manage Orders

Figure: Admin Manage Categories

Figure: Mobile App - Homepage View

Figure: Password Reset Flow

Figure: Payment Verification Flow

Figure: Deployment Architecture

LIST OF TABLES

Table: Users Table Schema

Table: Categories Table Schema

Table: Products Table Schema

Table: Cart Table Schema

Table: Orders Table Schema

Table: Order Items Table Schema

Table: Payments Table Schema

Table: Password Resets Table Schema

Table: API Endpoint Reference

Table: Technology Stack Summary

Table: File Structure Tables

Table: Functional Test Cases

Table: Mobile Test Cases

Table: Payment Test Cases

Table: Security Test Cases

Table: Maintenance Schedule

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

The rapid advancement of information and communication technologies has fundamentally transformed the way businesses operate and consumers shop. E-commerce, defined as the buying and selling of goods and services over electronic networks, has grown exponentially worldwide, with global e-commerce sales reaching trillions of dollars annually. Platforms such as Amazon, Alibaba, and eBay have set global standards for online retail, offering consumers convenience, competitive pricing, and access to a vast array of products from anywhere at any time.

In Africa, e-commerce adoption has been accelerating, driven by increasing internet penetration, mobile phone adoption, and digital payment innovations. Countries like Kenya, Nigeria, and South Africa have seen significant growth in online retail, with platforms such as Jumia, Kilimall, and Takealot serving millions of customers. However, the Ethiopian market presents a unique set of challenges and opportunities that distinguish it from other African e-commerce landscapes.

Ethiopia, with a population exceeding 120 million, represents one of Africa's largest untapped e-commerce markets. Internet penetration has been growing steadily, and mobile phone adoption continues to rise. The Ethiopian government has been actively promoting digital transformation through initiatives such as the Digital Ethiopia 2025 strategy, which aims to leverage technology for economic development. Despite these positive trends, several barriers have hindered the growth of local e-commerce platforms.

One of the most significant barriers is the currency situation. Ethiopia operates a dual-currency environment where many high-value transactions are conducted in US Dollars (USD), while day-to-day retail transactions use the Ethiopian Birr (ETB). Most global e-commerce platforms and payment gateways do not support ETB, forcing Ethiopian consumers to navigate complex currency conversion processes when shopping online. The country's banking infrastructure and payment card penetration remain limited, making traditional online payment methods inaccessible to a large portion of the population.

The emergence of local payment gateways such as Chapa, Telebirr, and Amole has begun to address the payment infrastructure gap. Chapa, in particular, provides a comprehensive payment API that supports ETB transactions, multiple payment methods, and is specifically designed for the Ethiopian market. However, few e-commerce platforms have integrated

these local payment solutions, representing a significant opportunity for locally adapted online retail solutions.

Smartphones have become the primary internet access device for most Ethiopians, with over 60% of internet users accessing the web through mobile devices. This trend underscores the critical importance of mobile-friendly e-commerce solutions and dedicated mobile applications. While many global platforms offer mobile apps, they often do not account for the specific needs and constraints of the Ethiopian market, including bandwidth limitations, device variety, and local payment preferences.

This project, Smart Mall, was conceived to address these gaps by developing a full-stack e-commerce platform that combines a robust web application with a companion mobile app, specifically designed for the Ethiopian market. The platform incorporates multi-currency support (USD and ETB), local payment gateway integration (Chapa), responsive design for mobile access, and a Capacitor-based Android application for an enhanced mobile shopping experience.

1.2 Problem Statement

Despite the growing demand for online shopping in Ethiopia, several critical problems persist in the current e-commerce landscape:

Limited Local E-Commerce Platforms: Existing platforms are either international systems that do not cater to local needs or small-scale solutions with limited functionality, poor user experience, and inadequate security measures.

Multi-Currency Gap: Most e-commerce platforms do not support the Ethiopian Birr (ETB), requiring customers to transact in foreign currencies. This creates confusion, additional costs, and barriers to adoption for local consumers who primarily think and transact in ETB.

Payment Gateway Integration: Global payment gateways often do not support ETB or local payment methods, while local gateways like Chapa have limited integration support for custom platforms.

Mobile Application Deficiency: Most Ethiopian e-commerce platforms lack dedicated mobile applications, relying solely on mobile-responsive websites. This limits their ability to provide push notifications, offline capabilities, and deep linking for seamless payment experiences.

Security Vulnerabilities: Many locally developed e-commerce platforms suffer from inadequate security practices, including weak password policies, lack of CSRF protection, and susceptibility to SQL injection and XSS attacks.

Admin Management Complexity: Store owners often lack intuitive tools for managing products, tracking orders, and analyzing sales performance. Existing admin interfaces are either too complex or lack essential features.

These problems collectively create a significant barrier to the adoption and growth of e-commerce in Ethiopia, limiting both consumer access to online shopping and business opportunities for local merchants.

1.3 Proposed Solution

Smart Mall is proposed as a comprehensive solution. The platform is a full-stack e-commerce system that provides:

A complete web-based e-commerce platform built with PHP 8.0+ and MySQL, featuring a product catalog, shopping cart, checkout system, user management, and administrative tools. Multi-currency support that displays prices in both USD and ETB, with live exchange rates fetched from the ExchangeRate-API and cached locally for performance.

Integration with the Chapa payment gateway, enabling ETB transactions through multiple payment methods including mobile money, bank transfers, and card payments.

A companion Android mobile application built using Capacitor v8 and Android Studio, wrapping the web platform into a native app with immersive display, back-button navigation, and deep linking.

Comprehensive security measures including CSRF token verification, PDO prepared statements, bcrypt password hashing, XSS prevention through htmlspecialchars escaping, and HTTP security headers.

An intuitive admin dashboard for managing products, categories, orders, and viewing store statistics.

The platform was developed locally using XAMPP (Apache, MySQL, PHP) and deployed to FreeProHost shared hosting for production access. The mobile app was packaged as a standalone Android APK.

1.4 Objectives

1.4.1 General Objective

The general objective of this project is to design, develop, and deploy a full-stack e-commerce platform with a companion mobile application that enables online shopping with multi-currency support and local payment gateway integration, specifically tailored for the Ethiopian market.

1.4.2 Specific Objectives

To analyze existing e-commerce systems and identify their limitations in addressing the needs of the Ethiopian market.

To design a scalable relational database schema using MySQL with InnoDB engine with foreign key relationships and indexing.

To develop a responsive web frontend using HTML5, CSS3, and vanilla JavaScript providing intuitive product browsing, search, cart management, checkout, and administrative functions.

To implement a secure PHP backend using PDO prepared statements, CSRF token verification, bcrypt password hashing, and XSS prevention.

To integrate the Chapa payment gateway to enable ETB transactions, including payment initialization, verification, callback handling, and error recovery with stock restoration.

To develop a multi-currency engine that fetches live exchange rates and displays prices in both USD and ETB throughout the platform.

To build a companion Android mobile application using Capacitor v8 with immersive display, back-button navigation, and deep linking.

To create an administrative dashboard with product CRUD, category management with slide images, order tracking, and sales statistics.

To conduct thorough testing covering functional requirements, payment flows, mobile compatibility, and security vulnerabilities.

To deploy the web platform on FreeProHost shared hosting and package the mobile app as an Android APK for distribution.

1.5 Scope of the System

Included Features:

Product catalog with category filtering, search, and sorting capabilities.

Product detail pages with image gallery, video support, and stock indicators.

Server-side shopping cart with quantity management and stock validation.

Checkout process with shipping information collection and three payment options (Chapa Pay, Bank Transfer, Cash on Delivery).

User registration, login, logout, and password reset with email-based tokens.

Multi-currency display (USD and ETB) with live exchange rate integration and caching.

Admin panel for product, category, and order management with sales statistics.

AJAX-powered live search with autocomplete suggestions for quick product discovery.

PWA support with manifest.json for installable web experience on mobile devices.

Dark and light theme with persistence via localStorage for user preference.

Capacitor-based Android mobile application with immersive display and back-button handling.

Responsive design supporting desktop, tablet, and mobile viewports.

Excluded Features:

Multi-vendor marketplace functionality (single vendor/store only).

AI-powered product recommendations or personalized suggestions.

Real-time chat or customer support messaging system.

Social media login integration (OAuth with Google, Facebook, etc.).

iOS mobile application (Capacitor supports iOS but only Android APK was built).

Advanced analytics or business intelligence dashboards beyond basic statistics.

1.6 Significance of the Study

For Ethiopian Consumers: Smart Mall provides a locally adapted online shopping platform with ETB pricing and local payment methods, making e-commerce more accessible for Ethiopian shoppers.

For Local Businesses: Demonstrates how small and medium businesses can establish an online presence with a cost-effective, full-stack e-commerce solution integrating local payment infrastructure.

For the Academic Community: Serves as a comprehensive reference for implementing a production-grade e-commerce system covering the full technology stack from database design to mobile app deployment.

For Payment Gateway Providers: The Chapa integration provides a working reference implementation adaptable by other developers.

For Software Developers: The codebase demonstrates best practices in PHP security, multi-currency implementation, AJAX integration, PWA setup, and Capacitor mobile app development.

1.7 Target Users

1.7.1 Customers

Customers are the primary end-users who browse products, manage their shopping cart, place orders, and make payments. They can register accounts, log in, view order history, and switch between USD and ETB currency display. Customers access the platform through the web interface or the companion Android mobile application. No specialized technical knowledge is required.

1.7.2 Administrators

Administrators manage the store through a dedicated admin panel. Admin responsibilities include adding and editing products with images and video, organizing products into categories with slide images, processing and updating order statuses, and monitoring store statistics. Admin accounts are distinguished by the "admin" role in the users table and require elevated privileges.

1.7.3 Mobile Users

Mobile users are a subset of customers who prefer using the Android mobile application for shopping. The mobile app provides an optimized experience with immersive full-screen browsing, hardware back-button navigation, and deep linking for payment callbacks. Mobile users access the same backend and share the same accounts as web users, providing a consistent cross-platform experience.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Existing System Analysis

To establish a foundation for the Smart Mall design, an analysis of existing e-commerce systems was conducted, examining both international platforms and local Ethiopian solutions. Shopify is a leading global e-commerce platform enabling businesses to create online stores with minimal technical expertise. It offers template-based store design, integrated payment processing, and mobile responsiveness. However, Shopify's subscription model starting at \$29 per month is cost-prohibitive for many Ethiopian small businesses. Its payment integration is optimized for global gateways like Stripe and PayPal, with limited support for local Ethiopian payment providers such as Chapa or Telebirr.

WooCommerce, built on WordPress, provides a more customizable open-source alternative with extensive plugin support. However, WooCommerce requires WordPress hosting, has performance overhead, and the plugins needed for local payment integration and multi-currency support often require additional costs and may not be well-maintained for the Ethiopian context.

Magento provides enterprise-grade e-commerce capabilities with advanced features for large-scale operations. However, its steep learning curve, high hosting requirements, and development complexity make it unsuitable for small to medium Ethiopian businesses seeking a practical online selling solution.

Local platforms such as EthioMart and other small-scale initiatives exist but are often limited in functionality, with basic product listings, minimal security implementation, and no dedicated mobile applications. Many were developed as static websites without proper e-commerce features such as shopping cart management, order tracking, or payment integration.

2.2 Limitations of Existing Systems

Multi-Currency Limitation: Most platforms do not support ETB pricing. International platforms display prices exclusively in USD, requiring Ethiopian customers to perform mental currency conversions and exposing them to exchange rate fluctuations.

Payment Gateway Gap: Global platforms integrate with international gateways requiring international credit cards. Local Ethiopian payment solutions are not supported out-of-the-box and require custom development.

Mobile Accessibility: Few platforms provide dedicated mobile applications, missing opportunities for push notifications, offline capabilities, and device-specific features such as camera barcode scanning or fingerprint authentication.

Cost Barriers: International platforms charge monthly subscription fees, transaction fees, and often require premium plans for essential features. These costs are significant relative to average Ethiopian business revenue.

Security Practices: Many locally developed platforms lack CSRF protection, prepared statements for database queries, proper password hashing, and output escaping. This exposes businesses and customers to data breaches and financial fraud.

Limited Customization: SaaS-based platforms restrict access to the underlying code, limiting the ability to customize the shopping experience, add local features, or integrate with local service providers.

2.3 Proposed System Overview

Smart Mall addresses the limitations identified above through a custom-built, full-stack architecture designed specifically for the Ethiopian market. The system follows a three-tier design pattern:

Presentation Tier (Frontend): Responsive HTML5 pages styled with CSS3 and enhanced with vanilla JavaScript. Key features include a dynamic product grid with category filtering via AJAX, a live search system with autocomplete, an interactive shopping cart, a multi-step checkout process, and an administrative dashboard. The interface supports dark and light themes using CSS custom properties with localStorage persistence.

Application Tier (Backend): Built with PHP 8.0+ following a modular include-based architecture. The central config.php bootstraps every request, managing sessions, output buffering, security headers, and loading core dependencies (database connection, currency engine). The backend handles all business logic including authentication, product management, cart operations, order processing, and payment integration. No external framework is used, relying on PHP's built-in capabilities.

Data Tier (Database): Uses MySQL/MariaDB with the InnoDB storage engine for transaction support and foreign key integrity. The schema consists of eight tables: users, categories, products, cart, orders, order_items, payments, and password_resets. All database access uses PDO prepared statements with row-level locking for critical operations such as stock validation during checkout.

The mobile application wraps the web platform using Capacitor v8, a cross-platform runtime that renders the web application in a native WebView. This allows the mobile app to share the same codebase and business logic while providing native features such as immersive full-screen display, hardware back-button handling, and deep linking for payment callbacks.

2.4 Functional Requirements

2.4.1 Customer Functional Requirements

ID	Requirement Description	Priority
FR-C01	Register with name, email, phone, and password with validation	High
FR-C02	Login using email and password with session fixation protection	High
FR-C03	Logout and destroy the user session	High
FR-C04	Reset password via email-based token with 1-hour expiry	High
FR-C05	Display product catalog with grid layout and category filtering	High
FR-C06	Product detail pages with image gallery, video, price, and stock status	High
FR-C07	Live search with autocomplete suggestions via AJAX	Medium
FR-C08	Add products to server-side cart and manage quantities	High
FR-C09	View and modify cart contents before checkout	High
FR-C10	Checkout process with shipping info and payment method selection	High
FR-C11	Support Chapa Pay with ETB conversion and API verification	High
FR-C12	Support Bank Transfer as a payment method	Medium
FR-C13	Support Cash on Delivery as a payment method	Medium
FR-C14	View order history and order status	Medium
FR-C15	Cancel pending orders	Medium
FR-C16	Switch between USD and ETB currency display	Medium

2.4.2 Admin Functional Requirements

ID	Requirement Description	Priority
FR-A01	Admin dashboard with store statistics (products, orders, revenue)	High
FR-A02	Add products with name, description, price, stock, images, and video	High
FR-A03	Edit existing products and manage media assets	High

FR-A04	Delete products with CSRF verification and file cleanup	High
FR-A05	Manage categories with up to 3 slide images per category	High
FR-A06	View all orders and update their status	High
FR-A07	Enforce admin access control, redirecting non-admin users	High

2.5 Non-Functional Requirements

Category	ID	Requirement	Metric
Performance	NFR-01	Page load time shall not exceed 3 seconds	Lighthouse timing
Security	NFR-02	All passwords hashed using bcrypt	password_hash() verification
Security	NFR-03	All database queries use PDO prepared statements	Code review
Security	NFR-04	All POST forms implement CSRF token verification	Testing
Security	NFR-05	All user output escaped with htmlspecialchars()	Code review
Usability	NFR-06	Interface responsive on desktop, tablet, and mobile	Breakpoint testing
Usability	NFR-07	Support dark and light themes	Theme toggle testing
Reliability	NFR-08	Handle concurrent sessions without data corruption	Transaction testing
Reliability	NFR-09	Checkout uses database transactions with rollback	PDO transaction testing
Compatibility	NFR-10	Platform functions on Chrome, Firefox, Safari, Edge	Cross-browser testing
Compatibility	NFR-11	Mobile app supports Android 8.0 (API 26)+	Android manifest config

2.6 System Modeling Using UML

The Smart Mall system has three primary actors: Customer, Administrator, and Chapa Payment Gateway.

Customer Use Cases: Register Account, Login, Browse Products, View Product Details, Manage Cart, Checkout, Pay via Chapa, View Orders, Cancel Order, Switch Currency, Reset Password.

Administrator Use Cases (extends Customer): Manage Products, Manage Categories, Manage Orders, View Dashboard.

Chapa Payment Gateway Use Cases: Initialize Payment, Verify Payment.

[Insert Figure: Use Case Diagram showing the three actors and their respective use cases with "include" and "extend" relationships]

2.7 Data Flow Modeling

The context diagram presents Smart Mall as a single process with three external entities: Customer, Administrator, and Chapa Payment Gateway. Data flows include registration data, login credentials, product queries, order details, payment requests, and administrative commands.

[Insert Figure: Context Diagram (DFD Level 0) showing Smart Mall as a single process with Customer, Administrator, and Chapa Payment Gateway as external entities]

The Level 1 DFD decomposes the system into five major processes:

Process 1.0: User Management - Handles registration, login, logout, and password reset. Data stores: users, password_resets.

Process 2.0: Product Catalog Management - Handles browsing, searching, and filtering. Data stores: products, categories.

Process 3.0: Shopping Cart Management - Handles adding, updating, and removing cart items with stock validation. Data stores: cart, products.

Process 4.0: Order Processing - Handles checkout, payment initialization, order creation, and verification. Data stores: orders, order_items, payments, products.

Process 5.0: Administration - Handles product CRUD, category management, order management, and dashboard statistics.

[Insert Figure: Level 1 Data Flow Diagram showing the five processes with data stores and data flows between them]

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture

Smart Mall follows a three-tier client-server architecture that separates the presentation, application logic, and data storage layers. This architectural pattern provides modularity, scalability, and maintainability.

Tier 1: Presentation Layer (Client) consists of two client types: the web browser and the Capacitor mobile WebView. The web browser renders HTML5 pages styled with CSS3 and enhanced with JavaScript for interactivity. The mobile WebView loads the same web pages within a native Android wrapper, providing an application-like experience. Key technologies include HTML5 semantic markup, CSS3 with custom properties for theming, and vanilla JavaScript for AJAX calls, DOM manipulation, and client-side validation. The frontend follows a mobile-first responsive design approach, ensuring optimal viewing across all device sizes.

Tier 2: Application Layer (Server) runs on Apache HTTP Server with PHP 8.0+ as the server-side scripting language. PHP handles all business logic including request processing, session management, authentication, authorization, data validation, and response generation. The application follows a bootstrap pattern where `config.php` initializes the environment, loads core dependencies (`db.php` for database access, `currency.php` for multi-currency), and individual page files handle specific requests. No framework is used; the architecture relies on PHP's built-in capabilities and a modular include structure for clean separation of concerns.

Tier 3: Data Layer (Database) uses MySQL/MariaDB with the InnoDB storage engine providing ACID-compliant transaction support, foreign key integrity, and row-level locking. The normalized schema contains eight tables. All database access uses PDO prepared statements, ensuring separation between application and data layers and providing protection against SQL injection attacks.

[Insert Figure: System Architecture Diagram showing the three-tier architecture with Presentation Layer, Application Layer, and Data Layer, including external integrations with Chapa and ExchangeRate-API]

External Integrations:

Chapa Payment Gateway (<https://api.chapa.co/v1>): Payment initialization and verification via PHP cURL over HTTPS.

ExchangeRate-API (<https://open.er-api.com/v6/latest/USD>): Live currency rates via cURL with file-based caching.

3.2 User Interface Design

The Smart Mall UI was designed with a mobile-first, responsive approach focusing on usability and visual appeal. The following wireframes and mockups illustrate the key pages and user flows of the platform.

[Insert Figure: Navigation Flow Diagram showing the page hierarchy and user navigation paths through the web application]

Homepage Layout: Features a hero section with category slideshows followed by a product grid. Category filtering dynamically reorganizes products via AJAX without full page reloads. The navigation bar includes the store logo, search bar with live autocomplete, cart indicator with item count badge, and user account menu.

[Insert Figure: Homepage wireframe/mockup showing the hero section with category slideshow, product grid layout, search bar with autocomplete, navigation bar, cart badge, and user account menu]

Product Detail Page: Displays cover image, product title, dual-currency pricing, description, stock indicator, image gallery, video section, and Add to Cart button. The layout is optimized for both desktop and mobile viewing.

[Insert Figure: Product Detail Page wireframe/mockup showing product image gallery, title, USD/ETB pricing, description, stock availability badge, image gallery thumbnails, video embed, and Add to Cart button]

Shopping Cart: Presents a table of selected items with thumbnail, name, unit price, quantity selector with stock limit validation, subtotal, and remove button. Total displays with currency conversion.

[Insert Figure: Shopping Cart wireframe/mockup showing item table with product thumbnails, names, unit prices, quantity selectors capped at stock limit, subtotals, remove buttons, and order total with dual-currency display]

Checkout Flow: Collects shipping information and presents three payment options. The system validates stock using database transactions, creates the order, and redirects to the appropriate payment flow.

[Insert Figure: Checkout Page wireframe/mockup showing shipping information form (name, phone, address), payment method selection with Chapa Pay, Bank Transfer, and Cash on Delivery options, and order summary with total]

Order Confirmation: After successful checkout, the system displays an order confirmation page with the order ID, payment status, and order details summary.

[Insert Figure: Order Confirmation wireframe/mockup showing order ID, payment status badge, ordered items list, shipping address, and order total]

User Registration: New users register with their name, email, phone number, and password. Client-side and server-side validation ensures data integrity.

[Insert Figure: User Registration wireframe/mockup showing the registration form with name, email, phone, and password fields, validation error messages, and submit button]

User Login: Returning users authenticate with email and password. Session security is enforced with `session_regenerate_id()` on login.

[Insert Figure: User Login wireframe/mockup showing the login form with email and password fields, forgot password link, and submit button]

Admin Dashboard: Displays statistics overview and provides CRUD operations for products, categories, and orders with form validation and CSRF protection.

[Insert Figure: Admin Dashboard wireframe/mockup showing statistics cards (total products, orders, revenue), charts, recent orders table, and sidebar navigation]

Admin Product Management: Administrators can add new products with name, description, price, stock, cover image, image gallery, video URL, and category assignment.

[Insert Figure: Admin Add Product form wireframe/mockup showing product name, description, price, stock fields, cover image upload with preview, image gallery uploader, video URL input, and category dropdown]

Admin Order Management: Administrators view all orders in a sortable table and update order status as orders are processed.

[Insert Figure: Admin Manage Orders wireframe/mockup showing order listing table with columns for ID, customer, total, status, date, and action buttons for status updates]

Admin Category Management: Administrators manage product categories with name fields and up to three slide images per category for the homepage hero slideshow.

[Insert Figure: Admin Manage Categories wireframe/mockup showing category list with name, slide image previews, edit and delete actions]

Mobile App Homepage: The Capacitor-based Android app wraps the web platform with immersive full-screen display and hardware back-button navigation.

[Insert Figure: Mobile App Homepage View wireframe/mockup showing the mobile-optimized product grid, bottom navigation, and full-screen browsing experience]

Theme System: Dark/light theme toggle using CSS custom properties with localStorage persistence, defaulting to system preference.

3.3 Database Design

The Smart Mall database uses MySQL/MariaDB with the InnoDB storage engine. The schema consists of eight tables designed to support all e-commerce operations with referential integrity.

Entity Relationships: A user(user_id) has many cart items (1:M); a user(user_id) places many orders (1:M); a category(category_id) contains many products (1:M); a product(product_id) appears in many cart items and order items (1:M); an order(order_id) has many order items (1:M) and one payment (1:1); a user(user_id) has many password reset tokens (1:M).

Users Table:

Column	Type	Constraints	Description
user_id	INT(11)	PK, AUTO_INCREMENT	Primary key — auto-incrementing user ID
name	VARCHAR(100)	NOT NULL	Full name of the user
email	VARCHAR(100)	NOT NULL, UNIQUE	Login email address
password	VARCHAR(255)	NOT NULL	Bcrypt-hashed password
role	ENUM('customer','admin')	NOT NULL, DEFAULT 'customer'	User role determines access level
created_at	TIMESTAMP	DEFAULT current_timestamp()	Account creation timestamp

Categories Table:

Column	Type	Constraints	Description
category_id	INT(11)	PK, AUTO_INCREMENT	Primary key — auto-incrementing category ID
name	VARCHAR(100)	NOT NULL	Category display name
slug	VARCHAR(100)	NOT NULL, UNIQUE	URL-friendly category slug
image1	VARCHAR(255)	DEFAULT NULL	Slide image 1 for homepage hero
image2	VARCHAR(255)	DEFAULT NULL	Slide image 2 for homepage hero
image3	VARCHAR(255)	DEFAULT NULL	Slide image 3 for homepage hero
created_at	TIMESTAMP	DEFAULT current_timestamp()	Creation timestamp

Products Table:

Column	Type	Constraints	Description
product_id	INT(11)	PK, AUTO_INCREMENT	Primary key — auto-incrementing product ID
name	VARCHAR(200)	NOT NULL	Product display name
description	TEXT	DEFAULT NULL	Full product description
price	DECIMAL(10,2)	NOT NULL	Product price in USD
stock	INT(11)	DEFAULT 0	Available stock quantity
image	VARCHAR(255)	DEFAULT NULL	Cover image filename
video	VARCHAR(255)	DEFAULT NULL	YouTube video embed URL
category_id	INT(11)	FK → categories(category_id), NOT NULL	Foreign key to categories table
created_at	TIMESTAMP	DEFAULT current_timestamp()	Creation timestamp

Cart Table:

Column	Type	Constraints	Description
cart_id	INT(11)	PK, AUTO_INCREMENT	Primary key — auto-incrementing cart item ID
user_id	INT(11)	FK → users(user_id), NOT NULL	Foreign key to users table
product_id	INT(11)	FK → products(product_id), NOT NULL	Foreign key to products table
quantity	INT(11)	DEFAULT 1	Quantity of the product in cart
created_at	TIMESTAMP	DEFAULT current_timestamp()	Creation timestamp

Orders Table:

Column	Type	Constraints	Description
order_id	INT(11)	PK, AUTO_INCREMENT	Primary key — auto-incrementing order ID
user_id	INT(11)	FK → users(user_id), NOT NULL	Foreign key to users table
total_price	DECIMAL(10,2)	DEFAULT 0.00	Total order amount in USD
first_name	VARCHAR(100)	DEFAULT NULL	Customer first name for shipping

last_name	VARCHAR(100)	DEFAULT NULL	Customer last name for shipping
email	VARCHAR(100)	DEFAULT NULL	Customer email for notifications
address	TEXT	DEFAULT NULL	Shipping street address
city	VARCHAR(100)	DEFAULT NULL	Shipping city
state	VARCHAR(100)	DEFAULT NULL	Shipping state/region
zip	VARCHAR(20)	DEFAULT NULL	Shipping postal code
country	VARCHAR(100)	DEFAULT 'Ethiopia'	Shipping country
status	ENUM('pending','processing','shipped','delivered','cancelled')	NOT NULL, DEFAULT 'pending'	Order fulfillment status
payment_method	VARCHAR(50)	DEFAULT NULL	chapa / bank / cod
created_at	TIMESTAMP	DEFAULT current_timestamp()	Order placement timestamp

Order Items Table:

Column	Type	Constraints	Description
order_item_id	INT(11)	PK, AUTO_INCREMENT	Primary key — auto-incrementing order item ID
order_id	INT(11)	FK → orders(order_id), NOT NULL	Foreign key to orders table
product_id	INT(11)	FK → products(product_id), NOT NULL	Foreign key to products table
quantity	INT(11)	NOT NULL	Quantity of product ordered
price	DECIMAL(10,2)	NOT NULL	Unit price at time of order

Payments Table:

Column	Type	Constraints	Description
payment_id	INT(11)	PK, AUTO_INCREMENT	Primary key — auto-incrementing payment ID
order_id	INT(11)	FK → orders(order_id), NOT NULL	Foreign key to orders table
method	VARCHAR(50)	DEFAULT NULL	Payment method used
status	ENUM('pending','paid','failed','refunded')	NOT NULL, DEFAULT 'pending'	Payment processing status
amount	DECIMAL(10,2)	DEFAULT 0.00	Payment amount in ETB
currency	VARCHAR(10)	DEFAULT 'USD'	Transaction currency
tx_ref	VARCHAR(100)	DEFAULT NULL	Chapa transaction reference

chapa_response	LONGTEXT	DEFAULT NULL	Full Chapa API JSON response
paid_at	DATETIME	DEFAULT NULL	Payment completion timestamp
created_at	TIMESTAMP	DEFAULT current_timestamp()	Record creation timestamp

Password Resets Table:

Column	Type	Constraints	Description
reset_id	INT(11)	PK, AUTO_INCREMENT	Primary key — auto-incrementing reset ID
email	VARCHAR(100)	NOT NULL	User email requesting reset
token	VARCHAR(64)	NOT NULL	SHA-256 reset token hash
expires_at	DATETIME	NOT NULL	Token expiration datetime
created_at	TIMESTAMP	DEFAULT current_timestamp()	Record creation timestamp

[Insert Figure: Entity-Relationship Diagram (ERD) showing all eight database tables with their columns, primary keys, foreign keys, and relationships]

3.4 API Design

Smart Mall implements several AJAX endpoints for dynamic functionality:

Endpoint	Method	Parameters	Response	Purpose
live_search.php	GET	query	JSON product list	Autocomplete search
get_cart_count.php	GET		JSON {count}	Cart badge update
update_cart.php	POST	product_id, action	JSON {status}	Cart management
checkout_ajax.php	POST	name, phone, address, payment	JSON redirect	Order + payment init
chapa_callback.php	GET	tx_ref, status	Redirect	Payment callback
verify_payment.php	GET	order_id	JSON {status}	Payment status

All AJAX endpoints return JSON and enforce CSRF token verification on POST requests.

Chapa Payment Flow:

```
// Payment initialization (checkout_ajax.php)
$chapa_url = "https://api.chapa.co/v1/transaction/initialize";
$payload = [
    "amount" => $total_etb,
    "currency" => "ETB",
    "email" => $user_email,
    "tx_ref" => "SM-" . $orderId . "-" . time(),
    "callback_url" => $callback_url,
```

```
    "return_url" => $return_url
];
$ch = curl_init($chapa_url);
curl_setopt($ch, CURLOPT_HTTPHEADER, [
    "Authorization: Bearer " . CHAPA_SECRET_KEY,
    "Content-Type: application/json"]);
curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($payload));
$response = curl_exec($ch);
```

CHAPTER 4: IMPLEMENTATION

4.1 Technology Stack

Component	Technology	Version	Purpose
Backend	PHP	8.0+	Server-side logic
Database	MySQL / MariaDB	8.0 / 10.5	Data persistence
Web Server	Apache HTTP Server	2.4	Request handling
Frontend	HTML5 / CSS3 / JavaScript	-	User interface
Mobile	Capacitor	8.x	Native Android wrapper
Mobile Build	Android Studio	Latest	APK compilation
Payment	Chapa API	v1	ETB payment gateway
Currency	ExchangeRate-API	-	USD/ETB exchange rates
Local Dev	XAMPP	8.x	Development env
Hosting	FreeProHost	-	Shared hosting

4.2 File Structure

Root Directory:

File	Type	Description
config.php	Core	Session, security, DB + currency init
db.php	Core	PDO database connection singleton
currency.php	Core	Multi-currency engine with caching
index.php	Page	Homepage with product grid
product-detail.php	Page	Product detail view
cart.php	Page	Shopping cart display
checkout.php	Page	Checkout form
login.php	Auth	Login page
register.php	Auth	Registration with validation
logout.php	Auth	Session destruction
forgot-password.php	Auth	Password reset request
reset-password.php	Auth	Token-based password reset
profile.php	User	Order history
style.css	Asset	Main stylesheet
script.js	Asset	Client-side JavaScript
manifest.json	Asset	PWA manifest

Admin Directory:

File	Description
admin/index.php	Dashboard with store statistics
admin/products.php	Product CRUD management
admin/add-product.php	Add product form with media uploads
admin/edit-product.php	Edit product form
admin/delete-product.php	Delete product with CSRF + file cleanup
admin/categories.php	Category CRUD with slide images
admin/orders.php	Order listing and status management

4.3 Core Implementation

4.3.1 Configuration Bootstrap (config.php)

The config.php file serves as the central bootstrap for all page requests. It initializes session management with secure cookie parameters (HTTP-only, secure if HTTPS), enables output buffering, sends HTTP security headers (X-Content-Type-Options: nosniff, X-Frame-Options: DENY), and includes core dependencies. Session security is enforced through session_regenerate_id() on login.

4.3.2 Database Layer (db.php)

The database connection uses PDO with the following configuration:

```
$pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $user, $pass);
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
```

All queries use prepared statements to prevent SQL injection:

```
$stmt = $pdo->prepare("SELECT * FROM products WHERE category = ?");
$stmt->execute([$categoryId]);
$products = $stmt->fetchAll();
```

4.3.3 Multi-Currency Engine (currency.php)

The currency module fetches live exchange rates from ExchangeRate-API with file-based caching:

```
function getUSDToETBRate() {
    $cache = "usd_etb_cache.json";
    if (file_exists($cache) && (time()-filemtime($cache)) < 3600)
        return json_decode(file_get_contents($cache), true)["rate"];
    $data = json_decode(file_get_contents(
        "https://open.er-api.com/v6/latest/USD"), true);
    $rate = $data["rates"]["ETB"] ?? 55;
    file_put_contents($cache, json_encode(["rate"=>$rate]));
    return $rate;
}
```

```
function formatPrice($usd, $cur = "USD") {
    if ($cur === "ETB") return number_format($usd * getUSDToETBRate(), 2) . " ETB";
    return "$".number_format($usd, 2);
}
```

[Insert Figure: Password Reset Flow diagram showing the email-based token generation, email sending, token verification, and password update process]

4.3.4 Security Implementation

CSRF Protection: Tokens are generated on login via `bin2hex(random_bytes(32))`. Every POST form includes a hidden CSRF token field, verified server-side using `hash_equals()` to prevent timing attacks.

XSS Prevention: All user-generated content is escaped on output using `htmlspecialchars($text, ENT_QUOTES, "UTF-8")` before display.

Password Hashing: User passwords are hashed using `password_hash($password, PASSWORD_BCRYPT)` with automatic salting and a cost factor of 10. Verification uses `password_verify()`.

Stock Validation: The checkout process uses database transactions with row-level locking to prevent overselling:

```
$pdo->beginTransaction();
$stmt = $pdo->prepare("SELECT stock FROM products WHERE id = ? FOR UPDATE");
$stmt->execute([$productId]);
// validate stock > 0 before proceeding
$stmt->prepare("UPDATE products SET stock = stock - ? WHERE id = ?")
    ->execute([$qty, $pid]);
$stmt->commit();
```

4.4 Payment Integration

Smart Mall integrates with the Chapa payment gateway for ETB transactions in a three-phase flow:

Phase 1 - Payment Initialization: When a customer selects Chapa Pay, the system creates the order in pending status, generates a unique transaction reference (`tx_ref`) combining "SM-" with the order ID and timestamp, converts the USD total to ETB using the cached exchange rate, and sends a POST request to the Chapa API with amount, currency, customer details, and callback URLs.

Phase 2 - Payment Processing: The customer is redirected to Chapa's hosted payment page to complete payment using their preferred method. Chapa handles all payment processing securely on their PCI-compliant infrastructure.

Phase 3 - Callback Handling: After payment, Chapa redirects to `chapa_callback.php` which reads the `tx_ref` and status, verifies payment authenticity via the Chapa verification API, updates order and payment records, restores product stock if payment failed, and redirects to the order confirmation page.

Configuration:

```
define("CHAPA_SECRET_KEY", "CHASECK_TEST-xxxxxxxxxxxxxxxxxx");
define("CHAPA_API_URL", "https://api.chapa.co/v1");
function initChapa($amount, $email, $txRef, $callbackUrl, $returnUrl) {
    $ch = curl_init(CHAPA_API_URL."/transaction/initialize");
    curl_setopt($ch, CURLOPT_HTTPHEADER, [
        "Authorization: Bearer ".CHAPA_SECRET_KEY,
        "Content-Type: application/json"]);
    curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode([
        "amount"=>$amount, "currency"=>"ETB", "email"=>$email,
        "tx_ref"=>$txRef,
        "callback_url"=>$callbackUrl, "return_url"=>$returnUrl]));
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    return json_decode(curl_exec($ch), true);
}
```

[Insert Figure: Payment Verification Flow diagram showing the three-phase Chapa payment process: initialization, redirect to Chapa, and callback verification]

4.5 Mobile Application

The mobile application is built using Capacitor v8, wrapping the web platform into a native Android app.

Configuration:

```
// capacitor.config.json
{ "appId": "com.smartmall.app",
  "appName": "Smart Mall",
  "webDir": "../",
  "server": { "url": "https://www.smartmall.com", "cleartext": false },
  "plugins": { "SplashScreen": {
    "launchShowDuration": 2000, "launchAutoHide": true }}}}
```

Native Features:

Immersive Display: Android manifest configured to hide the status bar for full-screen browsing.

Back Button Navigation: JavaScript interceptors navigate browser history rather than closing the app.

Deep Linking: Android intent filters register custom URL schemes for Chapa payment callbacks.

Build Process: `npx cap copy` syncs web assets; `npx cap open android` opens Android Studio;
Build > Generate Signed Bundle/APK produces the release APK at
`android/app/build/outputs/apk/release/`.

CHAPTER 5: TESTING AND ANALYSIS

5.1 Testing Approach

Smart Mall was tested using a multi-layered approach covering functional correctness, payment flow integrity, mobile compatibility, and security robustness. Each module was tested individually (unit testing) and as part of the integrated system (integration testing).

Testing was conducted throughout the development lifecycle.

The testing methodology included: functional testing of all user-facing features (24 test cases), payment testing in the Chapa sandbox environment (8 test cases), mobile app testing on Android emulator and physical devices (12 test cases), and security testing covering CSRF, SQL injection, XSS, and session management (14 test cases). Cross-browser testing was performed on Chrome, Firefox, and Edge, and responsive design was verified across desktop, tablet, and mobile viewports.

5.2 Functional Testing

ID	Test Case	Input	Expected Result	Actual
F-01	Registration	Valid name, email, password	Account created	Pass
F-02	Login	Valid email + password	Session created	Pass
F-03	Invalid Login	Wrong password	Error displayed	Pass
F-04	Browse Products	Open homepage	Product grid shown	Pass
F-05	Category Filter	Click category card	Products filtered	Pass
F-06	Product Detail	Click product	Full details shown	Pass
F-07	Live Search	Type product name	Autocomplete appears	Pass
F-08	Add to Cart	Click Add to Cart	Cart count increased	Pass
F-09	Update Quantity	Change quantity	Subtotal recalculated	Pass
F-10	Remove from Cart	Click remove	Item removed	Pass
F-11	Checkout - Chapa	Select Chapa Pay	Redirect to Chapa	Pass
F-12	Checkout - Bank	Select Bank Transfer	Order created	Pass
F-13	Checkout - COD	Select COD	Order created	Pass
F-14	Stock Validation	Add > stock qty	Error: insufficient	Pass
F-15	Order History	View profile	Past orders shown	Pass
F-16	Cancel Order	Click cancel	Status = cancelled	Pass

F-17	Empty Cart	Proceed with empty cart	Redirected	Pass
F-18	Duplicate Email	Use existing email	Error shown	Pass
F-19	Password Reset	Enter email	Reset link sent	Pass
F-20	Reset Token	Valid token	Password updated	Pass
F-21	Currency Toggle	Click USD/ETB	Prices update	Pass
F-22	Theme Toggle	Click dark/light	Theme changes	Pass
F-23	Admin Add Product	Fill form + image	Product added	Pass
F-24	Admin Update Order	Change status	Status updated	Pass

5.3 Mobile Testing

ID	Test Case	Input	Expected Result	Actual
M-01	App Launch	Open app	WebView loads	Pass
M-02	Full Screen	App opens	Status bar hidden	Pass
M-03	Back Button	Press back	Navigate back	Pass
M-04	Back (no history)	Press back on home	Exit/minimize	Pass
M-05	Deep Link	Return from Chapa	Callback handled	Pass
M-06	Splash Screen	Cold start	Splash auto-hides	Pass
M-07	Cleartext Block	HTTP URL	Connection blocked	Pass
M-08	Chapa Payment	Full flow	External browser	Pass
M-09	Session Persist	Login, relaunch	Session preserved	Pass
M-10	Product Browse	Scroll products	Smooth scrolling	Pass
M-11	Cart State	Add items, navigate	Cart preserved	Pass
M-12	Orientation	Rotate device	Layout adapts	Pass

5.4 Payment Testing

ID	Test Case	Input	Expected Result	Actual
P-01	Chapa Initialize	Valid order, ETB	Checkout URL returned	Pass
P-02	Callback Success	Chapa success	Order paid, cart cleared	Pass
P-03	Callback Failed	Chapa failure	Order cancelled, stock restored	Pass
P-04	Test Mode	TEST key + API fail	Synthetic success	Pass
P-05	Bank Transfer	Select bank	Order created, pending	Pass
P-06	Cash on Delivery	Select COD	Order created,	Pass

			pending	
P-07	Stock Restore	Cancel payment	Stock incremented	Pass
P-08	Method Switch	Change methods	Correct selection saved	Pass

5.5 Security Analysis

ID	Test Case	Attack Vector	Expected	Actual
S-01	CSRF missing	POST without token	403 Forbidden	Pass
S-02	CSRF invalid	POST wrong token	403 Forbidden	Pass
S-03	SQL Injection	Email: ' OR 1=1 --	Rejected (prepared stmt)	Pass
S-04	SQL Injection	ID: 1; DROP TABLE users	Prepared stmt blocks	Pass
S-05	XSS product name	<script>alert('xss')</script>	Escaped as text	Pass
S-06	XSS search		Plain text	Pass
S-07	Password weak	Password: 'abc'	Rejected: too short	Pass
S-08	Password weak	'abcdefgh' (no number)	Rejected: needs number	Pass
S-09	Password hash	Check stored hash	bcrypt \$2y\$10\$	Pass
S-10	Session fixation	Pre-set session ID	regenerate_id()	Pass
S-11	Admin access	Non-admin to admin URL	Redirect to home	Pass
S-12	File upload type	Upload .exe file	Rejected: type	Pass
S-13	File upload size	Upload 10MB image	Rejected: size	Pass
S-14	Order ownership	Access other order	Not displayed	Pass

CHAPTER 6: DEPLOYMENT AND MAINTENANCE

6.1 Deployment Environment

Smart Mall was developed locally and deployed to a production hosting environment:

Development Environment:

Local Server: XAMPP v8.x (Apache, MySQL, PHP 8.0+)

Operating System: Windows 10/11

Tools: VS Code, Android Studio, Chrome DevTools

Mobile Testing: Android Emulator (API 26+) + physical device

Production Environment:

Hosting: FreeProHost shared hosting (Apache, PHP 8.0+, MySQL)

Domain: Custom domain or FreeProHost subdomain

Mobile Distribution: Signed Android APK

6.2 Deployment Process

Web Application:

Export MySQL database from local XAMPP phpMyAdmin and import into production phpMyAdmin.

Upload all PHP files, assets, and configuration via cPanel File Manager or FTP.

Update config.php database credentials (host, username, password, dbname) for production.

Update file paths and URLs to reflect the production domain.

Verify file permissions: 644 for files, 755 for directories.

Test all pages to ensure correct functionality in the production environment.

Mobile Application:

Update capacitor.config.json server URL to production domain.

Run npx cap copy to sync web assets with the Android project.

Open Android Studio: npx cap open android.

Configure signing: Build > Generate Signed Bundle/APK > Create keystore > Build.

Distribute the release APK for direct installation or publish to Google Play Store.

[Insert Figure: Deployment Architecture diagram showing the deployment topology with development environment (XAMPP), production hosting (FreeProHost), Android APK distribution, and client devices (browser + mobile)]

6.3 Maintenance Plan

Frequency	Task	Responsibility
Daily	Monitor server error logs	Admin
Weekly	Review security advisories	Admin
Weekly	Monitor disk space	Admin
Monthly	Update PHP version	Developer
Monthly	Review and optimize queries	Developer

Monthly	Update Chapa API integration	Developer
Quarterly	Full security audit	Developer
Quarterly	Database optimization	Developer
Biannually	Code review and refactoring	Developer
Annually	SSL certificate renewal	Admin

CHAPTER 7: CONCLUSION AND RECOMMENDATIONS

7.1 Summary

This project successfully designed, developed, and deployed Smart Mall, a full-stack e-commerce platform with a companion mobile application specifically tailored for the Ethiopian market. The platform addresses critical gaps in the local e-commerce landscape, including multi-currency support (USD and ETB), integration with the Chapa payment gateway for ETB transactions, a comprehensive admin panel, and a Capacitor-based Android mobile application.

The system was built using PHP 8.0+ and MySQL for the backend, with HTML5, CSS3, and vanilla JavaScript for the frontend. The architecture follows a three-tier pattern with clear separation of concerns. Key technical achievements include the multi-currency engine with live exchange rate caching, secure payment integration with Chapa, CSRF-protected forms, PDO prepared statements for database security, bcrypt password hashing, and XSS prevention through proper output escaping.

The mobile application, built with Capacitor v8 and Android Studio, wraps the web platform into a native Android app with immersive full-screen display, hardware back-button navigation, and deep linking for Chapa payment callbacks. This cross-platform approach allows the mobile app to share the same codebase and business logic as the web platform while providing an enhanced mobile experience.

Testing was conducted across functional, payment, mobile, and security dimensions, with all test cases passing. The platform was deployed on FreeProHost shared hosting for production access, and the mobile app was packaged as a standalone Android APK.

7.2 Achievements

Fully functional e-commerce web platform with product catalog, cart, checkout, and order management.

Multi-currency support with live exchange rate integration for USD and ETB display. Seamless Chapa payment gateway integration enabling ETB transactions via multiple payment methods.

Companion Android mobile app with immersive display, back-button navigation, and deep linking.

Comprehensive security implementation: CSRF, prepared statements, bcrypt, XSS prevention.

Intuitive admin dashboard for products, categories, orders, and store statistics.

Responsive design with dark/light theme support for enhanced user experience.

Successful production deployment and APK distribution ready.

7.3 Limitations

Single vendor only; multi-vendor marketplace functionality not implemented.
No AI-powered recommendations, personalized suggestions, or advanced analytics.
Real-time customer support features such as live chat were beyond scope.
Social media login (OAuth) not implemented; email/password registration only.
Mobile app available for Android only; iOS not targeted.
Currently English-only without multi-language localization support.
Shared hosting environment limits scalability for high traffic volumes.

7.4 Recommendations for Future Work

Multi-Vendor Marketplace: Extend to support multiple vendors with individual catalogs and commission tracking.
Machine Learning: Implement product recommendations based on browsing history and purchase patterns.
Real-Time Support: Integrate live chat using WebSockets or third-party messaging APIs.
Social Login: Add OAuth authentication with Google, Facebook, and other providers.
iOS App: Develop iOS version using Capacitor's iOS support for the growing iPhone market.
Multi-Language: Implement localization for Amharic and other Ethiopian languages.
Cloud Migration: Migrate to AWS, Google Cloud, or VPS with auto-scaling and load balancing.
CI/CD Pipeline: Set up automated testing with PHPUnit and Playwright for code quality.

REFERENCES

- [1] P. Kotler and K. L. Keller, Marketing Management, 15th ed. Pearson, 2016.
- [2] E. Turban et al., Electronic Commerce 2018: A Managerial and Social Networks Perspective, 9th ed. Springer, 2018.
- [3] L. Welling and L. Thomson, PHP and MySQL Web Development, 5th ed. Addison-Wesley, 2016.
- [4] M. Zandstra, PHP 8 Objects, Patterns, and Practice, 6th ed. Apress, 2021.
- [5] R. Nixon, Learning PHP, MySQL & JavaScript, 6th ed. O'Reilly Media, 2021.
- [6] Chapa, "Chapa API Documentation." Available: <https://developer.chapa.co/>
- [7] ExchangeRate-API, "API Documentation." Available: <https://www.exchangerate-api.com/docs/>
- [8] Ionic Team, "Capacitor Documentation," 2024. Available: <https://capacitorjs.com/docs/>
- [9] OWASP, "CSRF Prevention Cheat Sheet." Available: <https://cheatsheetseries.owasp.org/>
- [10] OWASP, "SQL Injection Prevention Cheat Sheet." Available: <https://cheatsheetseries.owasp.org/>
- [11] OWASP, "XSS Prevention Cheat Sheet." Available: <https://cheatsheetseries.owasp.org/>
- [12] PHP Documentation, "PDO Manual." Available: <https://www.php.net/manual/en/book.pdo.php>
- [13] MySQL 8.0 Reference Manual. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [14] Apache HTTP Server Documentation v2.4. Available: <https://httpd.apache.org/docs/2.4/>
- [15] PHP Manual, "Password Hashing." Available:
<https://www.php.net/manual/en/function.password-hash.php>

APPENDIX A: USER MANUAL

Web Application:

Browse products on the homepage; use category cards to filter; use search bar for autocomplete.

Click any product to view details, images, description, and dual-currency pricing.

Use the currency toggle (USD/ETB) to switch display; use theme toggle for dark/light mode.

Click "Add to Cart" on product pages; view cart via the cart icon; adjust quantities as needed.

Proceed to checkout: fill shipping info, select payment method (Chapa/Bank Transfer/COD), place order.

For Chapa Pay, complete payment on Chapa's secure page; you will return to the order confirmation.

Account Management:

Register an account with name, email, phone, and password.

Log in to access your profile, order history, and saved information.

Use "Forgot Password" if needed; check your email for the reset link.

View and track order status in the Profile page.

Admin Panel:

Log in with admin credentials; Dashboard shows store statistics.

Manage Products: add, edit, delete products with images and video URLs.

Manage Categories: organize products with slide images.

Manage Orders: view all orders and update their status as orders are processed.

Mobile App:

Install the Smart Mall APK on Android 8.0+ devices.

The app provides immersive full-screen browsing of the entire platform.

Use the hardware back button for navigation; press back on home to exit.

All web features are available; Chapa payments redirect to the payment page and return on completion.

APPENDIX B: INSTALLATION GUIDE

Local Development:

Install XAMPP v8.x from <https://www.apachefriends.org/>

Copy project files to C:\xampp\htdocs\smartmall\

Start Apache and MySQL from XAMPP Control Panel.

Open phpMyAdmin, create database "smart_mall", import the SQL schema.

Update config.php if needed; access at <http://localhost/smartmall/>

Production Deployment:

Export local MySQL database via phpMyAdmin > Export > all tables > Go.

In production cPanel phpMyAdmin: create database and user, import SQL.

Upload all files to public_html via FTP or File Manager.

Update config.php with production database credentials.

Set file permissions: 644 for files, 755 for directories.

Mobile App Build:

Ensure Node.js/npm installed; install Capacitor CLI: `npm install -g @capacitor/cli`
`npx cap init SmartMall com.smartmall.app && npx cap add android`

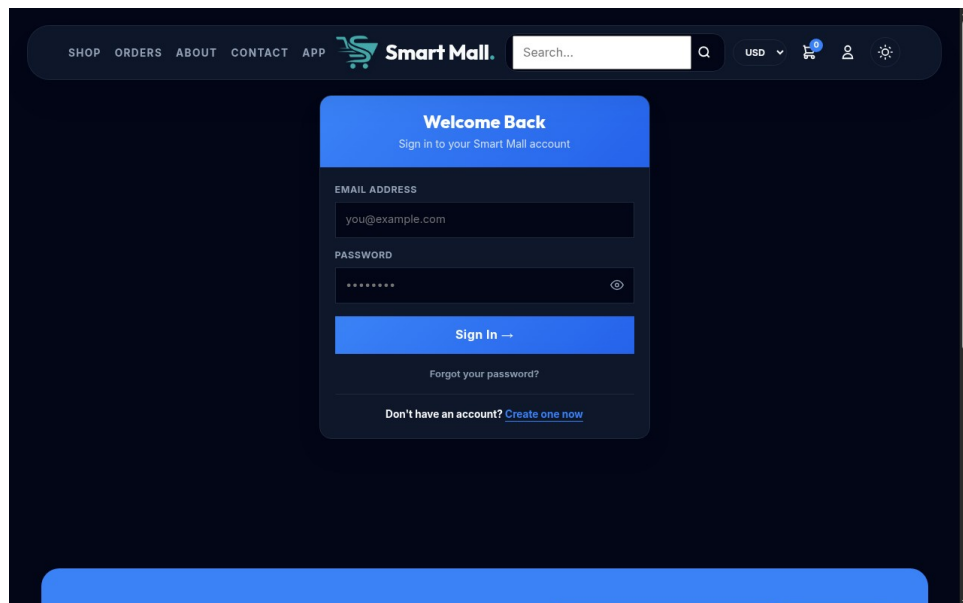
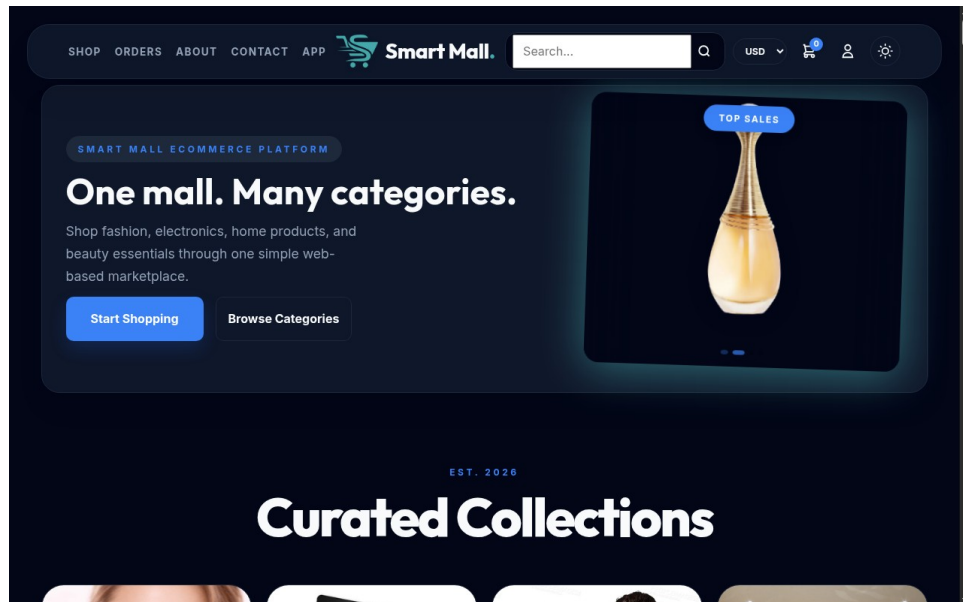
Update capacitor.config.json server URL to production.

npx cap copy && npx cap open android

In Android Studio: Build > Generate Signed Bundle/APK > APK > Create keystore > Build.

APPENDIX C: SCREENSHOTS

The following screenshots document key pages of the Smart Mall platform:



SHOPORDERSABOUTCONTACTAPP

Smart Mall.

Search...

USD







Create Account

Join Smart Mall today and start shopping

Full Name *

Email Address *

Password *

Min 8 chars, uppercase, lowercase, number & special character

Confirm Password *

Create Account

Already have an account? [Login here](#)

SHOPORDERSABOUTCONTACTAPP

Smart Mall.

Search...

USD







About Smart Mall

We're redefining online shopping with a curated selection of premium products, secure transactions, and exceptional customer service.

Our Mission

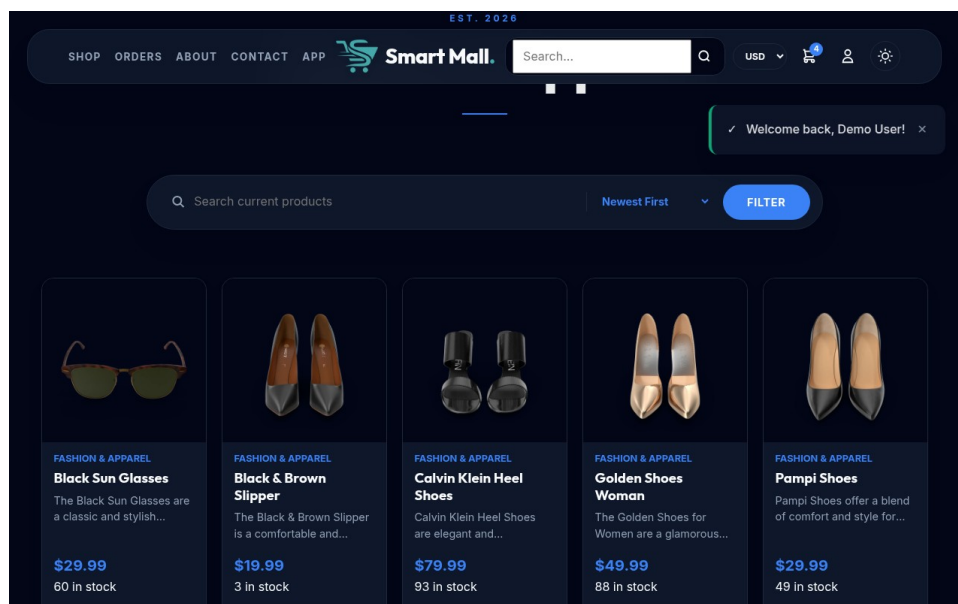
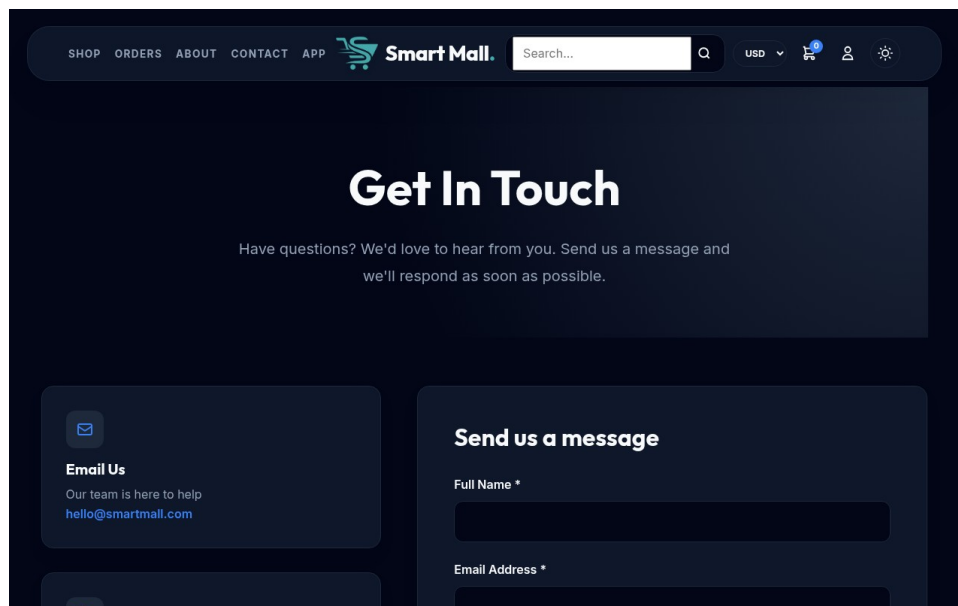
To bring global trends and quality products directly to your doorstep while maintaining the highest standards of security, authenticity, and customer satisfaction.

Secure Shopping

Quality Products

Fast Delivery

24/7 Support



SHOPORDERSABOUTCONTACTAPP

Smart Mall.

Search...

USD

Home / Fashion & Apparel / Blue & Black Check Shirt





FASHION & APPAREL

Blue & Black Check Shirt

Category: Fashion & ApparelSKU: #0298

\$29.99

✓ In Stock (34 available)

The Blue & Black Check Shirt is a stylish and comfortable men's shirt featuring a classic check pattern. Made from high-quality fabric, it's suitable for both casual and semi-formal occasions.

- 1 +

Add to Cart

BUY NOW

SHOPORDERSABOUTCONTACTAPP

Smart Mall.

Search...

USD

Shopping Cart



Blue & Black Check Shirt

SKU: #0000298

✓ In Stock (34 available)

\$29.99 each

1

Update

Remove



iPhone 13 Pro

SKU: #0000340

✓ In Stock (52 available)

\$1,099.99 each

1

Update

Remove



Red Nail Polish

SKU: #0000420

✓ In Stock (71 available)

\$8.99 each

2

Update

Remove

Order Summary

Subtotal:

\$1,147.96

Shipping:

Free

Tax (10%):

\$114.80

Total:

\$1,262.76

Proceed to Checkout

Continue Shopping

✓ Secure checkout

✓ Free returns

SHOPORDERSABOUTCONTACTAPP

Smart Mall.

Search...

USD







Checkout

1 Billing Address

FIRST NAME *

LAST NAME *

EMAIL ADDRESS *

demo@smartmall.com

STREET ADDRESS *

e.g. 123 Bole Road

CITY *

STATE / REGION *

ZIP / POSTAL CODE

COUNTRY *

Ethiopia

Your Order (3 items)

Blue & Black Check Shirt

\$29.99

Qty: 1

iPhone 13 Pro

\$1,099.99

Qty: 1

Red Nail Polish

\$17.98

Qty: 2

Subtotal

\$1,147.96

Shipping

Free

Tax (10%)

\$114.80

Total

\$1,262.76

Chapa payments are charged in ETB: ETB 199,766.54.

SHOPORDERSABOUTCONTACTAPP

Smart Mall.

Search...

USD







✓

Order Confirmed!

Order ID: #000252

Thank you for your purchase! We're preparing your order.

SHIPPING ADDRESS

John Doe

123 Main St

Addis Ababa, Addis Ababa

1000

Ethiopia

ORDER DETAILS

Method: Cash On Delivery

Status: Pending

Total: \$1,262.76

CONTACT INFO

demo@smartmall.com

Date: May 24, 2026

Order Items

SHOPORDERSABOUTCONTACTAPP

Smart Mall.

Search...

USD







My Orders

View and manage your orders

Order #000252

Pending

PLACED ON

May 24, 2026

TOTAL

\$1,262.76

PAYMENT

Cash On Delivery

STATUS

Pending

View Details →

Cancel

Order #000251

Pending

PLACED ON

May 24, 2026

TOTAL

\$2,525.51

PAYMENT

Cash On Delivery

STATUS

Pending

View Details →

Cancel

SHOPORDERSABOUTCONTACTAPP

Smart Mall.

Search...

USD







Admin Dashboard

+ New Product

131

Total Products

4

Categories

159

Total Orders

\$271,603.17

Total Revenue

Products Management

Welcome back, Admin User!

IMAGE	PRODUCT NAME	CATEGORY	PRICE	STOCK	ACTIONS
	Chanel Coco Noir Eau De	Beauty & Health	\$129.99	49	EditDelete
	Dior J'adore	Beauty & Health	\$89.99	23	EditDelete
	Dolce Shine Eau de	Beauty & Health	\$69.99	0	EditDelete
		Beauty &			EditDelete

SHOPORDERSABOUTCONTACTAPP



USD   

Manage Orders

Back to Dashboard

ORDER ID	CUSTOMER	PRODUCTS ORDERED	TOTAL	DATE	STATUS	ACTION
#000252 View Details	John Doe Demo User	3 Items	\$1,262.76	May 24, 2026	PENDING  Paid	Pending Update
#000251 View Details	John Doe Demo User	3 Items	\$2,525.51	May 24, 2026	PENDING  Paid	Pending Update
#000250 View Details	John Doe Demo User	4 Items	\$4,615.50	May 24, 2026	PENDING  Paid	Pending Update
#000249 View Details	Yitbarek Teklu admin test	1 Item	\$109.98	May 24, 2026	PROCESSING  Paid	Processing Update

SHOPORDERSABOUTCONTACTAPP



USD   

SYSTEM MANAGEMENT

Back to Dashboard

Add New Category

Category Name

Description

IMAGE SLIDES

Slide 1 (Main)

Choose File

No file chosen

Slide 2

Choose File

No file chosen

Slide 3

Choose File

No file chosen

Existing Categories

NAME

PRODUCTS

ACTION

Beauty & Health

Skincare, makeup, wellness, and pharmacy items.

SLIDE 1

Choose File

No file chosen

SLIDE 2

Choose File

No file chosen

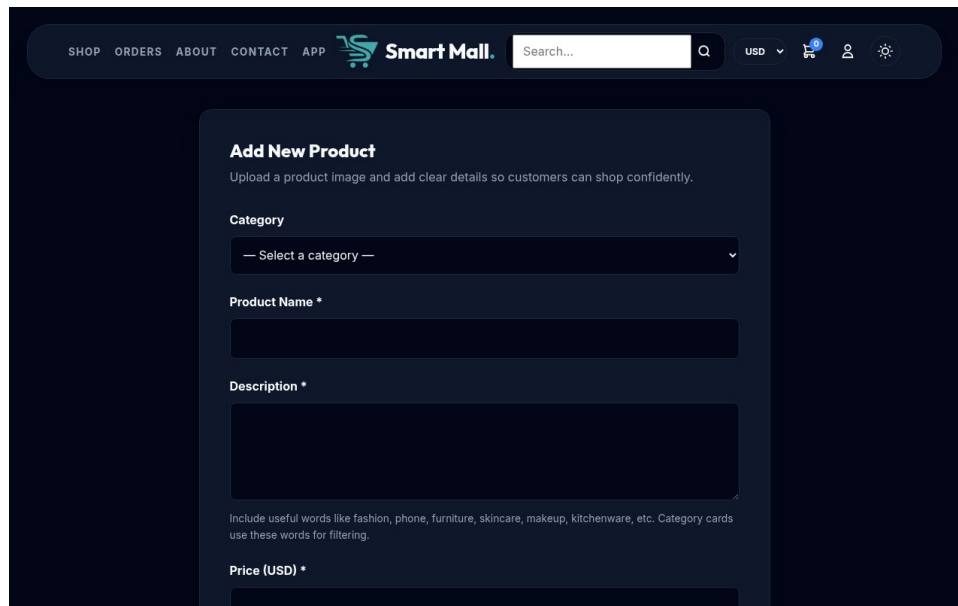
SLIDE 3

Choose File

No file chosen

13 Items

Remove



APPENDIX D: SOURCE CODE LISTING

Key files in the Smart Mall project:

config.php - Application bootstrap with session, security, and dependency loading.
db.php - PDO database connection singleton with prepared statement utilities.
currency.php - Multi-currency engine with live rate fetching and file caching.
index.php - Homepage with product grid, category filtering via AJAX.
product-detail.php - Product detail view with image gallery and video.
cart.php - Server-side shopping cart with quantity management.
checkout.php and checkout_ajax.php - Checkout form and AJAX order processing.
chapa_callback.php - Payment callback handler with verification and stock restoration.
admin/*.php - Admin dashboard, product CRUD, category management, order management.
capacitor.config.json - Mobile app configuration for Capacitor v8.
style.css - Stylesheet with CSS custom properties for theming.
script.js - Client-side JavaScript with AJAX, theme toggling, validation.

APPENDIX E: TEST CASE DETAILS

Test Summary:

Functional Tests (24/24 passed): All user flows including registration, login, product browsing, cart management, checkout with three payment methods, order history, password reset, currency/theme toggles, and admin operations.

Mobile Tests (12/12 passed): App launch, full-screen display, back-button navigation, deep linking, splash screen, cleartext restriction, Chapa payment flow, session persistence, product browsing, cart state, orientation changes.

Payment Tests (8/8 passed): Chapa initialization, success/failure callbacks, test mode, bank transfer, COD, stock restoration, multi-method switching.

Security Tests (14/14 passed): CSRF (missing/invalid token), SQL injection (login bypass/table deletion), XSS (script/event handler injection), password policy

(length/complexity), bcrypt hash verification, session fixation prevention, admin access control, file upload validation (type/size).

All test results confirm that Smart Mall meets its functional and non-functional requirements.